

Table of contents

Serie 1	1
Part 1: Resolution Methods	1
Methods for Tautologies and Validity	1
Methods for First-Order Logic	2
Methods for Satisfaction Problems	2
Review of Fundamental Logical Laws	3
Implication Laws	3
De Morgan's Laws	3
Distributive Laws	3
Associativity and Commutativity Laws	3
Identity and Complement Laws	3
Part 2: Illustrative Examples	4
Example 1: Transformation by Rewriting (Modus Ponens)	4
Example 2: Counterexample Search	4
Example 3: Formalization in First-Order Logic	5
Example 4: Modeling Uniqueness Constraints	5
Example 5: Systematic Reasoning (Box Problem)	6
Example 6: Conjunctive Normal Form	6
Methods Diagram	7
Synthesis	8

Serie 1

Part 1: Resolution Methods

Methods for Tautologies and Validity

1. Complete Truth Table

Exhaustive method that evaluates all possible variable combinations to determine if a formula is always true.

2. Transformation by Rewriting

Systematic use of logical laws to simplify or transform formulas until an obvious form is obtained.

3. Counterexample Search

To prove that a formula is **not** a tautology, search for a variable assignment that makes the formula false.

4. Conjunctive Normal Form (CNF)

Transformation of a formula into a conjunction of disjunctions of literals, allowing easy validity checking.

5. Case Analysis

Systematic examination of all possible scenarios for a variable or subformula.

Methods for First-Order Logic

6. Identification of Predicates and Constants

Precise definition of predicates, functions, and constants needed to model a natural language statement.

7. Guided Translation by Logical Structure

Use of logical connectives ($\forall$, $\exists$, $\Rightarrow$, $\wedge$, $\vee$, $\neg$) to reflect the structure of the statement.

8. Quantifier Management

Correct placement of quantifiers to capture variable scope.

9. Modeling Uniqueness Constraints

Use of quantifier combinations and equality to express uniqueness.

Methods for Satisfaction Problems

10. Propositional Formalization

Transformation of a concrete problem into propositional variables and logical constraints.

11. Satisfiability by Systematic Reasoning

Examination of constraints and search for consistent assignments by eliminating impossible cases.

Review of Fundamental Logical Laws

Implication Laws

- **Implication elimination:** $p \Rightarrow q \equiv \neg p \vee q$
- **Contrapositive:** $p \Rightarrow q \equiv \neg q \Rightarrow \neg p$
- **Negation of implication:** $\neg(p \Rightarrow q) \equiv p \wedge \neg q$

De Morgan's Laws

- $\neg(p \wedge q) \equiv \neg p \vee \neg q$
- $\neg(p \vee q) \equiv \neg p \wedge \neg q$

Distributive Laws

- $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
- $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$

Associativity and Commutativity Laws

- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
- $(p \vee q) \vee r \equiv p \vee (q \vee r)$
- $p \wedge q \equiv q \wedge p$
- $p \vee q \equiv q \vee p$

Identity and Complement Laws

- $p \vee \text{False} \equiv p$
 - $p \wedge \text{True} \equiv p$
 - $p \vee \neg p \equiv \text{True}$
 - $p \wedge \neg p \equiv \text{False}$
-

Part 2: Illustrative Examples

Example 1: Transformation by Rewriting (Modus Ponens)

Formula: $(p \wedge (p \Rightarrow q)) \Rightarrow q$

Application of methods:

- Use of implication elimination
- Application of De Morgan's laws
- Use of distributivity
- Simplification by complement laws

Demonstration:

$$\begin{aligned} & (p \wedge (p \Rightarrow q)) \Rightarrow q \\ \equiv & \neg(p \wedge (p \Rightarrow q)) \vee q \quad (\text{elimination of } \Rightarrow) \\ \equiv & \neg(p \wedge (\neg p \vee q)) \vee q \quad (\text{elimination of } \Rightarrow) \\ \equiv & (\neg p \vee \neg(\neg p \vee q)) \vee q \quad (\text{De Morgan}) \\ \equiv & (\neg p \vee (p \wedge \neg q)) \vee q \quad (\text{De Morgan}) \\ \equiv & ((\neg p \vee p) \wedge (\neg p \vee \neg q)) \vee q \quad (\text{distributivity}) \\ \equiv & (\text{True} \wedge (\neg p \vee \neg q)) \vee q \quad (\text{complement}) \\ \equiv & (\neg p \vee \neg q) \vee q \\ \equiv & \neg p \vee (\neg q \vee q) \\ \equiv & \neg p \vee \text{True} \\ \equiv & \text{True} \end{aligned}$$

Example 2: Counterexample Search

Formula: $(q \vee r \Rightarrow p \wedge \neg s) \wedge (s \vee p \Rightarrow (\neg r \Rightarrow q)) \Rightarrow \neg(s \vee r)$

Application of methods:

- Analysis of implication structure
- Search for assignment making premise true and conclusion false
- Consistency verification

Demonstration:

1. **For the conclusion to be false:** $\neg(s \vee r)$ must be false, so $s \vee r$ must be true. Choice:
 $s = \text{False}, r = \text{True}$

2. **For the premise to be true:**

- First part: $q \vee r \Rightarrow p \wedge \neg s$ With $r = \text{True}$, $q \vee r$ is true. For the implication to be true, $p \wedge \neg s$ must be true. With $s = \text{False}$, $\neg s$ is true, so p must be true.
- Second part: $s \vee p \Rightarrow (\neg r \Rightarrow q)$ With $p = \text{True}$, $s \vee p$ is true. $\neg r$ is false (since r is true), so $\neg r \Rightarrow q$ is true (false premise).

3. **Found assignment:** $p = \text{True}$, q arbitrary, $r = \text{True}$, $s = \text{False}$ This assignment makes the premise true and the conclusion false, so the formula is not a tautology.

Example 3: Formalization in First-Order Logic

Statement: “Every person who works in computer science has written at least one program”

Application of methods:

- Identification of predicates: $\text{CS}(x)$, $\text{P}(y)$, $W(x, y)$
- Analysis of structure: “for all x , if $\text{CS}(x)$ then there exists y such that...”
- Guided translation by logical structure

Formalization:

$$\forall x (\text{CS}(x) \Rightarrow \exists y (\text{P}(y) \wedge W(x, y)))$$

Example 4: Modeling Uniqueness Constraints

Statement: “Each program was written by a unique person who works in computer science”

Application of methods:

- Expression of existence: $\exists y$
- Expression of uniqueness: $\forall z ((\text{CS}(z) \wedge W(z, x)) \Rightarrow z = y)$
- Combination with conjunction

Formalization:

$$\forall x (\text{P}(x) \Rightarrow \exists y (\text{CS}(y) \wedge W(y, x) \wedge \forall z ((\text{CS}(z) \wedge W(z, x)) \Rightarrow z = y)))$$

Example 5: Systematic Reasoning (Box Problem)

Problem: Find which box contains the treasure with contradictory statements

Application of methods:

1. **Propositional formalization:**

- Variables T_i : treasure in box i
- Variables B_i : statement on box i is true

2. **Constraint translation:**

- $B_1 \Leftrightarrow \neg T_1$
- $B_2 \Leftrightarrow \neg T_2$
- $B_3 \Leftrightarrow T_1$
- Exactly one B_i true
- Exactly one T_i true

3. **Case analysis:**

- Case 1: T_1 true $\rightarrow$ contradiction with B_2
- Case 2: T_3 true $\rightarrow$ contradiction with uniqueness of B_i
- Case 3: T_2 true $\rightarrow$ all constraints satisfied

Conclusion: The treasure is in B_2 .

Example 6: Conjunctive Normal Form

Formula: $(\neg q \Rightarrow \neg p) \Rightarrow ((\neg q \Rightarrow p) \Rightarrow q)$

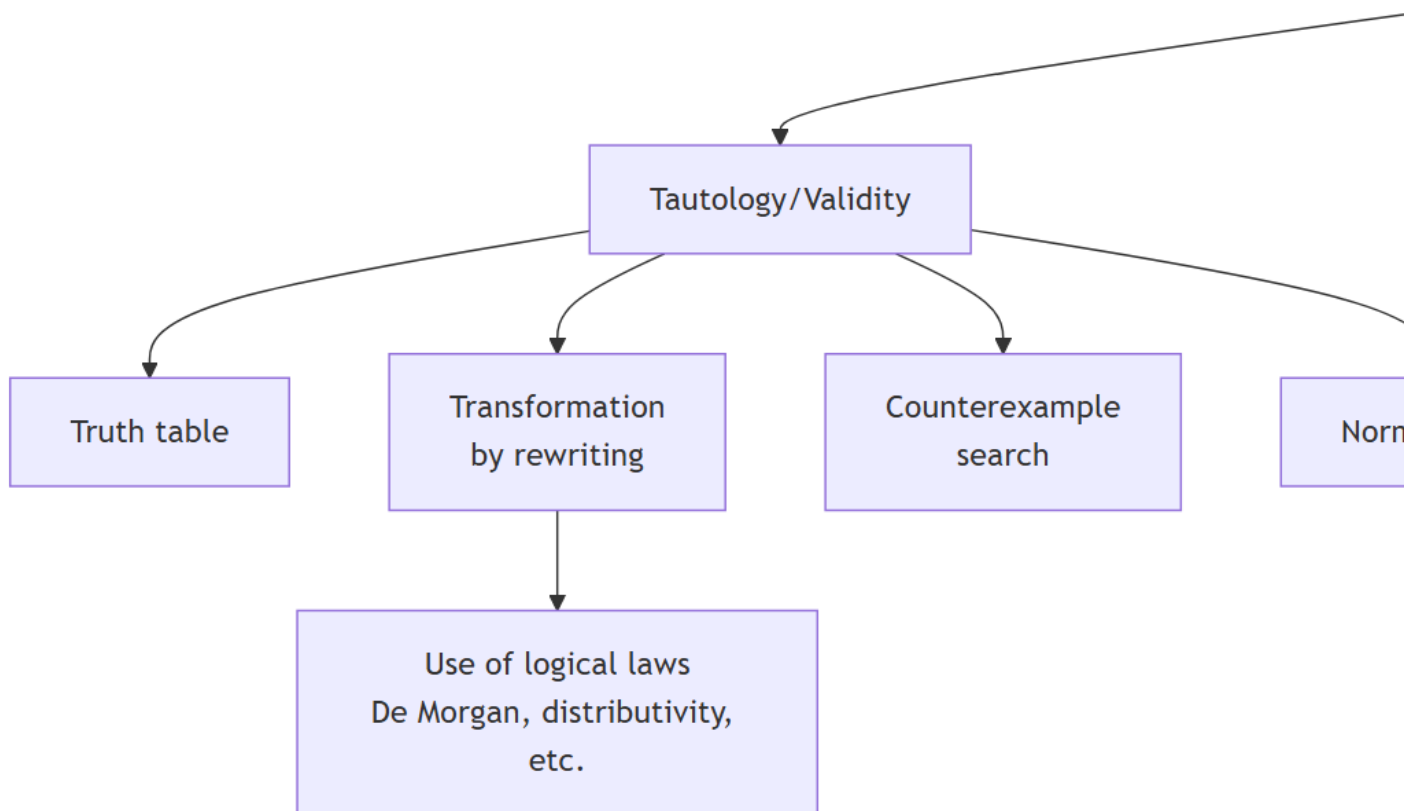
Application of methods:

- Transformation to CNF
- Observation that each clause contains a literal and its negation
- Validity conclusion

Demonstration (excerpt):

After transformations, we obtain a CNF where each clause is of the form $(p \vee q \vee \neg q)$ or similar, therefore always true.

Methods Diagram



Synthesis

The presented methods cover the main aspects of problem solving in propositional and first-order logic. Mastering these methods, combined with a good knowledge of fundamental logical laws, allows addressing a wide variety of problems in software modeling and verification.

Key points:

1. Always start by understanding the logical structure of the problem
2. Choose the method adapted to the problem type (validation, modeling, satisfaction)
3. Systematically apply logical laws in transformations
4. Verify the consistency of found solutions
5. Clearly document each step of the reasoning